```
# Step 1: Import libraries
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Step 2: Load MNIST handwritten digit dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training images:", X_train.shape)
print("Testing images:", X_test.shape)

# Step 3: Display one image
plt.imshow(X_train[0], cmap="gray")
plt.title("Actual Digit: " + str(y_train[0]))
plt.axis("off")
plt.show()

# Step 4: Normalize pixel values
# Original pixel values are between 0 and 255
# Convert them into values between 0 and 1

X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
# Step 5: Add channel dimension
# CNN expects: height, width, channel

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Step 6: Build the CNN model

model = Sequential([

    # Detect simple features such as edges and curves
    Conv2D(
        filters=32,
        kernel_size=(3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    # Reduce the image size
    MaxPooling2D(pool_size=(2, 2)),

    # Convert the feature maps into one-dimensional form
    Flatten(),

    # Learn important combinations of features
    Dense(64, activation="relu"),
```

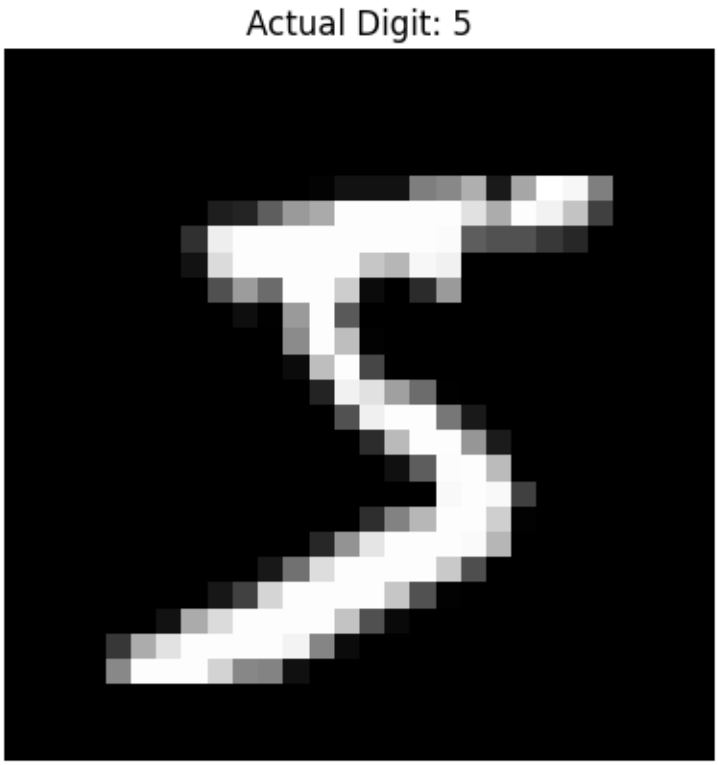
```
    # Output layer for digits 0 to 9
    Dense(10, activation="softmax")
])

# Step 7: Compile the model

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Step 8: Display model architecture

model.summary()

# Step 9: Train the model

model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)
```

```
# Step 10: Test the model

test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", test_accuracy)

# Step 11: Predict one test image

prediction = model.predict(X_test[0:1])

predicted_digit = prediction.argmax()

plt.imshow(X_test[0].reshape(28, 28), cmap="gray")
plt.title(
    "Actual: " + str(y_test[0]) +
    " | Predicted: " + str(predicted_digit)
)
plt.axis("off")
plt.show()
```

OUT PUT:

Training images: (60000, 28, 28)

Testing images: (10000, 28, 28)



Model: "sequential_1"

Layer (type)	**Output Shape**	**Param #**
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0

| dense_2 (Dense) | (None, 64) | 346,176 |

| dense_3 (Dense) | (None, 10) | 650 |

Total params: 347,146 (1.32 MB)

Trainable params: 347,146 (1.32 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

1688/1688 **35s** 20ms/step - accuracy: 0.9483 - loss: 0.1799 - val_accuracy: 0.9805 - val_loss: 0.0686

Epoch 2/5

1688/1688 **38s** 18ms/step - accuracy: 0.9817 - loss: 0.0600 - val_accuracy: 0.9847 - val_loss: 0.0528

Epoch 3/5

1688/1688 **31s** 18ms/step - accuracy: 0.9881 - loss: 0.0397 - val_accuracy: 0.9882 - val_loss: 0.0476

Epoch 4/5

1688/1688 **29s** 17ms/step - accuracy: 0.9906 - loss: 0.0288 - val_accuracy: 0.9847 - val_loss: 0.0590

Epoch 5/5

1688/1688 **29s** 17ms/step - accuracy: 0.9938 - loss: 0.0200 - val_accuracy: 0.9873 - val_loss: 0.0539

313/313 **2s** 6ms/step - accuracy: 0.9840 - loss: 0.0503

Test Accuracy: 0.984000027179718

WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at

0x7939612e1440> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 81ms/step